ASSIGNMENT-20.5

Name: V. Vyshnavi

H NO:2303A51968

Batch-14

Task 1 – Password Storage Vulnerability Check Task:

Analyze an AI-generated signup script for insecure password storage.

Expected Output:

• A secure registration script with hashed password storage.

PROMPT:

Generate a Python user registration script that stores username and password in a file or database without using hashing

CODE

Untited-1.pi 16.py

16.py > ...

```python
#Generate a Python user registration script that stores usernames and passwords in a text fi
def register_user():
    username = input("Enter a username: ")
    password = input("Enter a password: ")

    with open("users.txt", "a") as file:
        file.write(f"{username}:{password}\n")

    print("User registered successfully!")
if __name__== "_main__":
    register_user()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS C:\Users\neera\OneDrive\Desktop\assignments & C:\Users/neera\AppData\Local\Python\pythoncore-3.14
64\python.exe "C:\Users\neera\OneDrive\Desktop\assignments\16.py"
Enter a username: Soumya
Enter a password: 1234
User registered successfully!

PS C:\Users\neera\OneDrive\Desktop\assignments
```

JUSTIFICATION:

The AI-generated script stores passwords in plain text, which is insecure because anyone with access to the file can easily read user credentials. This

creates a major security risk, especially if sensitive data is involved. To improve security, passwords should be converted into a hashed format using libraries like hashlib or bcrypt.

Task 2 – Hardcoded Secrets Detection Task:

Evaluate AI-generated code for hardcoded API keys or credentials.

Instructions:

- Ask AI to generate a script that connects to an external API.
- Identify if API keys/passwords are written directly in code.
- Refactor using environment variables or config files.

Expected Output:

- A secure script with secrets managed properly.

PROMPT:

Generate a Python script that connects to an external API using a hardcoded API key or credentials in the code.

JUSTIFICATION:

The AI-generated script include API keys or credentials directly in the code, which is insecure because anyone accessing the code can misuse them. Hardcoding secrets increases the risk of unauthorized access and data breaches. To improve security, sensitive information should be stored in environment variables or configuration files. This ensures that credentials are protected and not exposed in the source code.

CODE:

```python
import requests
api_key = "YOUR_HARDCODED_API_KEY_HERE"

# Using JSONPlaceholder as a public API for testing.
# It does not require an API key for basic GET requests.
api_url = "https://jsonplaceholder.typicode.com/posts"

# Define headers, including the API key (common for many APIs)
headers = {
    "Authorization": f"Bearer {api_key}", # Common for APIs using Bearer tokens
    "Content-Type": "application/json"
}

# Define any parameters or data for the request
params = {
    "userId": "1" # Example: Fetch posts by user ID 1
}

try:
    # Make the GET request
    response = requests.get(api_url, headers=headers, params=params)

    # Check if the request was successful (status code 200)
    response.raise_for_status()

    # Print the response content (e.g., JSON)
    print("API Response:")
    print(response.json())

except requests.exceptions.HTTPError as http_err:
    print(f"HTTP error occurred: {http_err}")
except requests.exceptions.ConnectionError as conn_err:
    print(f"Connection error occurred: {conn_err}")
except requests.exceptions.Timeout as timeout_err:
    print(f"Timeout error occurred: {timeout_err}")
except requests.exceptions.RequestException as req_err:
    print(f"An error occurred: {req_err}")
```

```
API Response:
[{'userId': 1, 'id': 1, 'title': 'sunt aut facere repellat provident occaecati excepturi optio reprehenderit', 'body': 'quia et suscipit\nsuscipit
```

Task 3 – File Upload Security Validation

Task:

Test an AI-generated file upload program for unsafe file handling.

Instructions:

- Prompt AI to generate a Flask file upload example.
- Check for vulnerabilities like uploading executable files.
- Add validation for file type, size, and secure filename handling.

Expected Output:

- A secure file upload implementation.

PROMPT:

Generate a simple Flask application that allows users to upload files without any validation or restrictions.

CODE

```python
import os
from flask import Flask, request, redirect, url_for, render_template

app = Flask(__name__)

# Define the upload folder. Create it if it doesn't exist.
UPLOAD_FOLDER = 'uploads'
if not os.path.exists(UPLOAD_FOLDER):
    os.makedirs(UPLOAD_FOLDER)
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER

@app.route('/')
def upload_form():
    return '''
    <!doctype html>
    <title>Upload New File</title>
    <h1>Upload New File</h1>
    <form method=post action="/upload" enctype=multipart/form-data>
      <input type=file name=file>
      <input type=submit value=Upload>
    </form>
    '''

@app.route('/upload', methods=['POST'])
def upload_file():
    if 'file' not in request.files:
        return redirect(request.url)
    file = request.files['file']
    if file.filename == '':
        return redirect(request.url)
    if file:
        filename = file.filename # No validation or sanitization!
        filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
        file.save(filepath)
        return f'File {filename} uploaded successfully to {UPLOAD_FOLDER}!'
    return redirect('/')
```

JUSTIFICATION:

The AI-generated Flask file upload program allows files to be uploaded without any validation, which can be dangerous. Users might upload harmful files like scripts or executables that could affect the system. There are no checks for file type, size, or filename, increasing security risks. To make it secure, validation should be added to allow only specific file types, limit file size, and use secure filenames. This helps prevent malicious uploads and protects the application.

To test this Flask application in Google Colab, you'll need to use a tool like `ngrok` to expose the local server running inside the Colab environment to the internet. Here's how you can do that:

1. **Install `pyngrok`:**

```
!pip install flask pyngrok
```

2. **Run the Flask app with `ngrok`:**

```
from pyngrok import ngrok
import threading
import time

# Kill any ngrok processes already running
ngrok.kill()

# Start ngrok tunnel
public_url = ngrok.connect(5000)
print(f" * ngrok tunnel available at: {public_url}")

# Function to run Flask app in a separate thread
def run_flask():
    app.run(port=5000)

# Start Flask app in a new thread
thread = threading.Thread(target=run_flask)
thread.daemon = True
thread.start()

# Give Flask a moment to start
time.sleep(3)
print(f" * Open the public URL to access your app: {public_url}")
```

After running the above cell, you will get a public URL. Click on that URL to access your Flask application and upload files. The files will be saved in the `uploads` folder in your Colab environment.

Task 4 – Command Injection Prevention

Task:

Analyze an AI-generated Python script that executes system commands.

Instructions:

- Ask AI to generate code using os.system() or subprocess.
- Identify command injection risks with user input.
- Fix using safe subprocess calls and input validation.

Expected Output:

- A secure command execution script.

PROMPT:

Generate a Python script that takes user input and executes it as a system command using os.system() or subprocess.

CODE

```python
import subprocess

# Get user input for the command
user_command = input("Enter a system command to execute (e.g., 'mkdir my_directory'): ")

print(f"Executing command: {user_command}")

try:
    result = subprocess.run(user_command, shell=True, capture_output=True, text=True, check=True

    print("\nCommand output:")
    print(result.stdout)
    if result.stderr:
        print("\nCommand error (stderr):")
        print(result.stderr)

except subprocess.CalledProcessError as e:
    print(f"\nError executing command: {e}")
    print(f"Stderr: {e.stderr}")
except FileNotFoundError:
    print(f"\nError: Command '{user_command.split()[0]}' not found.")
except Exception as e:
    print(f"\nAn unexpected error occurred: {e}")
```

```
Enter a system command to execute (e.g., 'mkdir my_directory'): mkdir --help
Executing command: mkdir --help

Command output:
Usage: mkdir [OPTION]... DIRECTORY...
Create the DIRECTORY(ies), if they do not already exist.

Mandatory arguments to long options are mandatory for short options too.
  -m, --mode=MODE   set file mode (as in chmod), not a=rwx - umask
  -p, --parents     no error if existing, make parent directories as needed
  -v, --verbose     print a message for each created directory
  -Z                   set SELinux security context of each created directory
                         to the default type
      --context[=CTX]  like -Z, or if CTX is specified then set the SELinux
                         or SMACK security context to CTX
      --help     display this help and exit
      --version  output version information and exit
```

JUSTIFICATION:

The AI-generated script executes user input directly as a system command, which is unsafe and can lead to command injection attacks. Malicious users can input harmful commands that may damage the system or access sensitive data. This makes the program highly vulnerable to security breaches. To fix this, user input should be validated and safer methods like subprocess with controlled arguments should be used. This ensures that only intended commands are executed and reduces security risks.

Task 5 – Secure Input Handling in Web Forms Task:

Check AI-generated web form code for improper input validation.

Instructions:

- Ask AI to generate an HTML + Flask feedback form.
- Identify missing validation and unsafe rendering.

Expected Output:

- A secure form resistant to injection attacks.

PROMPT:

Generate a basic HTML and Flask feedback form that accepts user input and displays it without validation or sanitization.

CODE

```
from flask import Flask, request, render_template_string

app = Flask(__name__)

HTML_FORM = '''
<!doctype html>
<title>Feedback Form</title>
<h1>Submit Your Feedback</h1>
<form method=post action="/submit_feedback">
  <label for="name">Name:</label><br>
  <input type="text" id="name" name="name" size="50"><br><br>
  <label for="email">Email:</label><br>
  <input type="text" id="email" name="email" size="50"><br><br>
  <label for="feedback_text">Your Feedback:</label><br>
  <textarea id="feedback_text" name="feedback_text" rows="5" cols="50"></textarea><br><br>
  <input type="submit" value="Submit Feedback">
</form>
'''

HTML_DISPLAY = '''
<!doctype html>
<title>Feedback Received</title>
<h1>Feedback Received</h1>
<p><strong>Name:</strong> {{ name }}</p>
<p><strong>Email:</strong> {{ email }}</p>
<p><strong>Feedback:</strong></p>
<pre>{{ feedback_text }}</pre>
<p><a href="/feedback">Submit more feedback</a></p>
'''

@app.route('/feedback')
def feedback_form():
    return render_template_string(HTML_FORM)

@app.route('/submit_feedback', methods=['POST'])
def submit_feedback():
    user_name = request.form.get('name', 'N/A')
    user_email = request.form.get('email', 'N/A')
    user_feedback = request.form.get('feedback_text', 'No feedback provided')

    # WARNING: Displaying input without sanitization! This is for demonstration only.
    return render_template_string(HTML_DISPLAY, name=user_name, email=user_email, feedback_text=user_feedback)
```

JUSTIFICATION:

The AI-generated script executes system commands directly using user input, which creates a serious security vulnerability. Attackers can inject malicious commands that may harm the system or access sensitive data. This lack of input validation makes the application unsafe and unreliable. To prevent this, safer methods like subprocess.run() with controlled arguments should be used instead of os.system(). Additionally, validating and restricting user input ensures that only intended commands are executed, improving overall security.